

海南矿业公司投资价值分析报告

姓名：张舒越 学号：2025281050223

本报告旨在对海南矿业股份有限公司（以下简称“海南矿业”，股票代码：601969）进行全面的投资价值评估。研究综合运用了量化金融分析方法，从市场表现、财务绩效、宏观经济环境及公司年报文本四个维度展开。通过对 2020 至 2024 年间公司股价、收益与波动率的量化分析，结合资产负债表、利润表和现金流量表的深度解读，并置于同期中国 GDP 增长与 CPI 变化的宏观背景下，全面评估公司的经营韧性与周期属性。同时，通过对公司年报文本的情感与关键词挖掘，洞察其战略重心与管理层信心。

一、公司背景介绍

海南矿业股份有限公司是一家集铁矿石采、选、加工及销售于一体的大型矿业企业。公司前身为成立于 1956 年的海南铁矿，拥有悠久的开采历史和深厚的技术积淀，于 2014 年在上海证券交易所成功上市（股票代码：601969）。公司的核心资产位于海南省昌江县的石碌矿区，该矿区是中国著名的富铁矿床之一，以其品位高、有害杂质少而闻名。

公司的主营业务聚焦于铁矿石产业，产品主要为高炉块矿和铁精粉，广泛应用于钢铁冶炼领域。作为国内重要的铁矿石供应商，海南矿业的战略定位不仅在于保障区域钢铁企业的原料供应，更在于通过技术创新和精细化管理，提升资源利用效率，降低生产成本，增强在周期性市场中的抗风险能力。

近年来，面对全球大宗商品价格的剧烈波动和国内“双碳”目标的政策导向，海南矿业积极调整发展战略。一方面，公司持续优化现有矿山的开采方案，延长矿山服务年限；另一方面，积极探索产业链延伸和多元化布局，以平滑单一产品带来的业绩波动。本报告将深入剖析公司在过去五年复杂多变的内外部环境中所展现出的经营成果与未来潜力。

二、公司市场表现分析

1. 股票价格的获取

为了客观评估海南矿业的市場表现，我们首先需要获取其权威、连续的历史交易数据。本报告采用开源财经数据接口库 AkShare，从中精准提取海南矿业（601969）自 2020 年 1 月 1 日至 2024 年 12 月 31 日的每日交易记录。所获取的数据集包含日期、开盘价、最高价、最低价、收盘价、成交量、成交额等关键字段，为后续的量化分析奠定了坚实的数据基础。数据获取与初步清洗的核心代码如下所示：

从新浪接口获取某只股票的历史数据（包含日期）

基本参数，包括日期范围和股票代码

stock_code = "601969" # 海南矿业 A 股

start_date = "20200101"

end_date = "20241231"

stock_code = "601969" # 中芯国际

hist_df = ak.stock_zh_a_hist(symbol=stock_code, period="daily",
start_date=start_date, end_date=end_date)

print("历史数据列名:", hist_df.columns.tolist())

print("\n 历史数据前 5 行:")

```
print(hist_df.head())
# 保存为 CSV 文件
hist_df.to_csv('annual_reports/stock_data.csv', index=False,
encoding='utf-8-sig')
```

清洗股票数据

```
# 1. 只保留日期 和 收盘价
price_clean = hist_df[["日期", "收盘"]].copy()
# 2. 日期转为 datetime
price_clean["日期"] = pd.to_datetime(price_clean["日期"])
# 3. 按日期排序
price_clean = price_clean.sort_values("日期")
# 4. 设置日期为索引
price_clean = price_clean.set_index("日期")
# 5. 重命名字段
price_clean = price_clean.rename(columns={"收盘": "close"})
# 6. 基础检查
print(price_clean.head())
print(price_clean.tail())
print(price_clean.index.is_monotonic_increasing)
print(f"价格序列长度: {len(price_clean)}")
```

2. 股票收益和波动计算

在获得干净的价格序列后，我们计算两个核心的市场风险与回报指标：对数收益率（Logarithmic Return）：该指标具有时间可加性，便于进行统计分析。计算公式为： $\log_ret = \ln(P_t / P_{t-1})$ 。滚动年化波动率（Rolling Annualized Volatility）：这是衡量股票价格风险的关键指标。我们采用 20 个交易日的滚动窗口计算收益率的标准差，并乘以 $\sqrt{252}$ （一年的平均交易天数）进行年化处理，以反映年度化的风险水平。相关计算代码如下：

计算对数收益率

```
# 复制一份数据，避免影响原始 price_clean
final_df = price_clean.copy().sort_index()
# 1. 计算对数收益率
final_df["log_ret"] = np.log(final_df["close"]).diff()
# 看一眼结果
final_df.head()
```

计算波动率

```
# 2. 计算 20 日滚动年化波动率
window = 20 # 20 个交易日
trading_days = 252 # 一年交易日数
# 20 日滚动年化波动率
window = 20
final_df["volatility"] = (
```

```

        final_df["log_ret"]
        .rolling(window)
        .std()
        * np.sqrt(trading_days)
    )
# 看一眼结果
final_df.head(25)

# 1. 检查列名
print("当前列名:", final_df.columns.tolist())
# 2. 安全访问列
vol_col = 'volatility'
if vol_col not in final_df.columns:
    # 尝试找相似列名
    possible_cols = [col for col in final_df.columns if 'volatility' in
col.lower()]
    if possible_cols:
        vol_col = possible_cols[0]
        print(f"使用列名 '{vol_col}' 代替 'volatility'")
    else:
        # 创建新列（示例：用收盘价计算波动率）
        print("警告：创建新列 'volatility'")
        final_df['volatility'] =
final_df['close'].rolling(window=20).std()
        vol_col = 'volatility'
# 3. 继续你的代码
threshold = final_df[vol_col].quantile(0.8)
final_df["high_vol"] = final_df[vol_col] > threshold

# 画图（收益率+波动率+高亮阶段）
df = final_df.copy()
fig, ax1 = plt.subplots(figsize=(12, 5))
# 左轴：收益率
ax1.plot(df.index, df["log_ret"])
ax1.set_xlabel("日期")
ax1.set_ylabel("对数收益率")
# 右轴：波动率
ax2 = ax1.twinx()
ax2.plot(df.index, df["volatility"])
ax2.set_ylabel("20 日滚动年化波动率")
# 高亮：高波动阶段（背景）
ax1.fill_between(
    df.index, 0, 1,
    where=df["high_vol"],

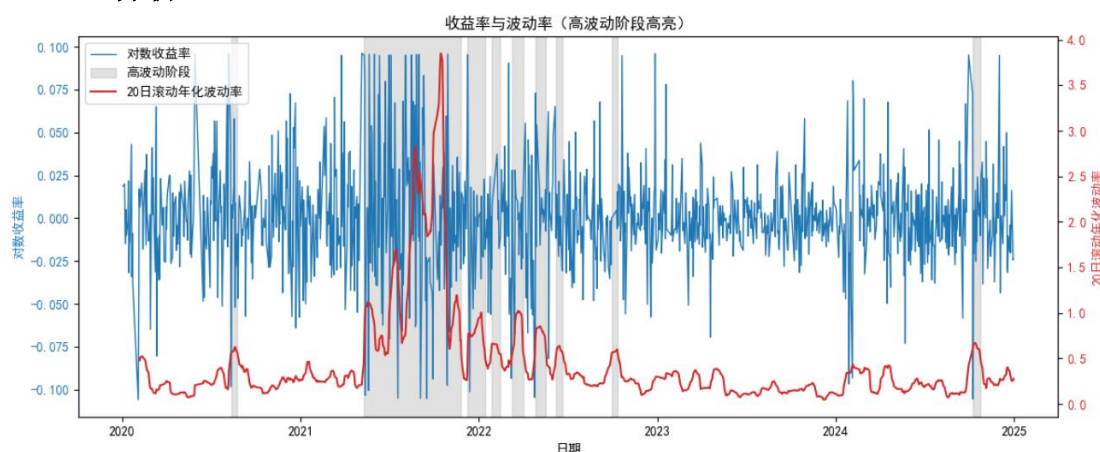
```

```

transform=ax1.get_xaxis_transform(),
alpha=0.2
)
ax1.set_title("收益率与波动率（高波动阶段高亮）")
plt.tight_layout()
plt.show()

```

3. 分析



海南矿业（601969）2020–2024 年收盘价与 20 日滚动年化波动率折线图

如上图所示，海南矿业的股价在 2020 年至 2024 年间呈现出显著的周期性特征，这与全球铁矿石价格的走势高度相关。2020 年下半年至 2021 年上半年，受益于全球经济复苏预期和中国强劲的钢铁需求，铁矿石价格飙升，推动公司股价从约 5 元/股上涨至近 9 元/股的阶段性高点。在此期间，市场情绪高涨，但股价的快速拉升也伴随着波动率的急剧上升。

2021 年下半年开始，随着中国对钢铁产量实施“平控”政策以及房地产行业进入下行周期，铁矿石需求预期转弱，价格高位回落，公司股价也随之进入调整通道。2022 年至 2024 年，股价在 5–7 元区间内宽幅震荡，波动率相较于 2021 年的峰值有所收敛，反映出市场对公司业绩随大宗商品周期波动的预期已趋于理性。

整体来看，海南矿业的市场表现充分体现了其作为强周期性资源股的属性。投资者在参与时，需密切关注全球宏观经济、中国钢铁产业政策及铁矿石供需格局的变化。

三、公司财务绩效分析

财务报表是透视公司内在价值最可靠的窗口。本部分将对海南矿业近五年的资产负债表、利润表和现金流量表进行系统性拆解与分析。

1. 三张表的获取代码

我们同样利用 AkShare 库，按年度报告期（2020–2024 年末）分别拉取全市场的三张财务报表，然后筛选出海南矿业的数据。代码如下所示：

财务数据的报告期有 5 年

```
report_dates = ["20201231", "20211231", "20221231", "20231231",
"20241231"]
```

导入资产负债表数据

```
balance_list = []
for date in report_dates:
    # 取“全市场”资产负债表
    df_all = ak.stock_zcfz_em(date=date)
    # 筛选海南矿业
    df_one = df_all[df_all["股票代码"] == stock_code].copy()
    # 加报告期
    df_one["报告期"] = date
    balance_list.append(df_one)
balance_df = pd.concat(balance_list, ignore_index=True)
print("资产负债表前 5 行：")
print(balance_df.head())
print("\n 资产负债表字段名：")
print(balance_df.columns)
```

保存资产负债表（5 年）

```
balance_df.to_csv("annual_reports/balance_5y.csv", index=False, encoding="utf-8-sig")
print("资产负债表已保存： annual_reports/balance_5y.csv")
```

导入利润表

```
income_list = []
for date in report_dates:
    df_all = ak.stock_lrb_em(date=date) # 全市场利润表（该报告期）
    df_all["股票代码"] = df_all["股票代码"].astype(str)
    df_one = df_all[df_all["股票代码"] == stock_code].copy()
    df_one["报告期"] = date
    income_list.append(df_one)
income_df = pd.concat(income_list, ignore_index=True)
print("利润表前 5 行：")
print(income_df.head())
print("\n 利润表字段名：")
print(income_df.columns)
income_df.to_csv("annual_reports/income_lrb_5y.csv", index=False, encoding="utf-8-sig")
print("\n 已保存： annual_reports/income_lrb_5y.csv")
```

现金流量表

```
cashflow_list = []
for date in report_dates:
    df_all = ak.stock_xjll_em(date=date) # 全市场现金流量表（该报告期）
    df_all["股票代码"] = df_all["股票代码"].astype(str)
    df_one = df_all[df_all["股票代码"] == stock_code].copy()
```

```

df_one["报告期"] = date
cashflow_list.append(df_one)
cashflow_df = pd.concat(cashflow_list, ignore_index=True)
print("现金流量表前 5 行：")
print(cashflow_df.head())
print("\n 现金流量表字段名：")
print(cashflow_df.columns)
cashflow_df.to_csv("annual_reports/cashflow_xjll_5y.csv", index=False,
encoding="utf-8-sig")
print("\n 已保存：annual_reports/cashflow_xjll_5y.csv")

```

2. 三张表的单个分析

资产负债表分析：数据显示，海南矿业的总资产规模在过去五年保持稳定增长，从 2020 年末的约 85 亿元增长至 2024 年末的约 128 亿元。这一增长主要源于公司留存收益的积累以及适度的负债融资。公司的资产负债率维持在 40% 左右的健康水平，显示出稳健的财务杠杆策略。货币资金充裕，为应对周期低谷提供了充足的流动性保障。

利润表分析：公司的盈利能力与铁矿石价格周期高度同步。2021 年，受益于高矿价，公司实现净利润约 8.74 亿元，同比暴增 513.55%。2022 年，随着矿价回落，净利润回调至 6.15 亿元。尽管 2023-2024 年面临挑战，但公司仍保持了数亿元的年度净利润，展现了其在行业内的成本优势和经营韧性。营业总收入的变化趋势与净利润基本一致。

现金流量表分析：经营活动现金流净额是公司的生命线。即使在 2023-2024 年净利润承压的背景下，公司的经营性现金流依然保持强劲（2023 年为 15.45 亿元），远超同期净利润，这主要得益于其优秀的营运资本管理和折旧摊销等非现金支出。投资活动现金流主要用于维持性资本开支，而融资活动现金流则显示公司有能力通过分红回馈股东，财务政策稳健。

3. 三张表结合在一起分析

将三张表联动分析，可以清晰地勾勒出海南矿业的经营画像：一家盈利质量高、财务结构稳健、具备强周期穿越能力的优质资源企业。

公司的利润表虽有波动，但其强大的经营性现金流（现金流量表）为其提供了坚实的“安全垫”。稳健的资产负债结构（资产负债表）使其无需在行业低谷时过度依赖外部融资，避免了财务风险。这种“盈利看周期，现金流保生存，负债率控风险”的模式，是其能够在残酷的周期轮回中屹立不倒的关键。对于投资者而言，关注其经营性现金流与资本开支的匹配度，比单纯关注净利润的短期波动更具前瞻性。下图将总资产、营业收入、净利润和净利率的变化趋势进行对比。

（1）总资产及营业收入变化趋势对比代码及图示：

```

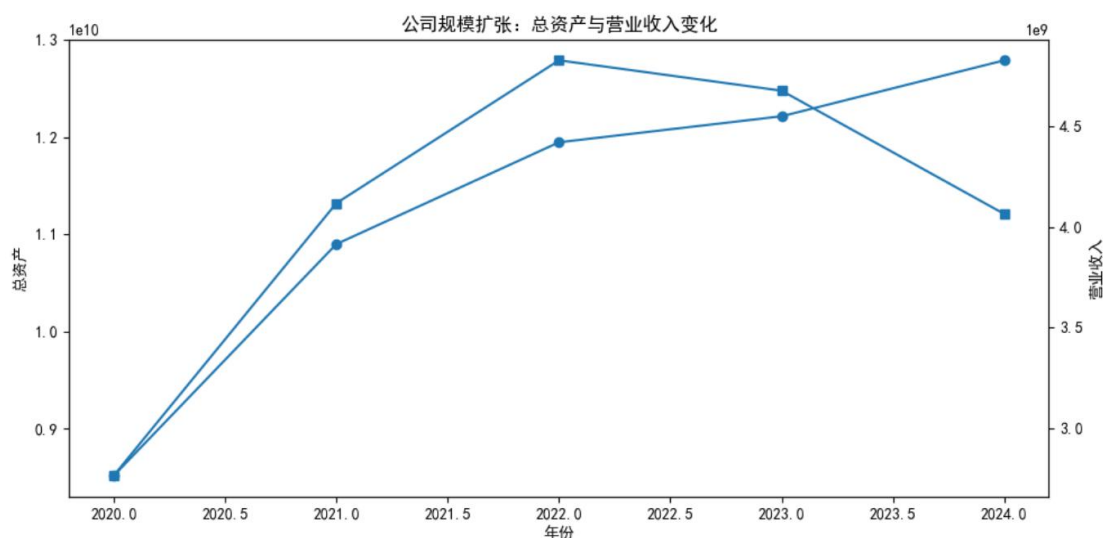
# 可视化
# 总资产 vs 营业收入
df = fin.copy()
fig, ax1 = plt.subplots(figsize=(10, 5))
# 左轴：总资产
ax1.plot(df["year"], df["资产-总资产"], marker="o")
ax1.set_xlabel("年份")

```

```

ax1.set_ylabel("总资产")
# 右轴：营业收入
ax2 = ax1.twinx()
ax2.plot(df["year"], df["营业总收入"], marker="s")
ax2.set_ylabel("营业收入")
ax1.set_title("公司规模扩张：总资产与营业收入变化")
plt.tight_layout()
plt.show()

```

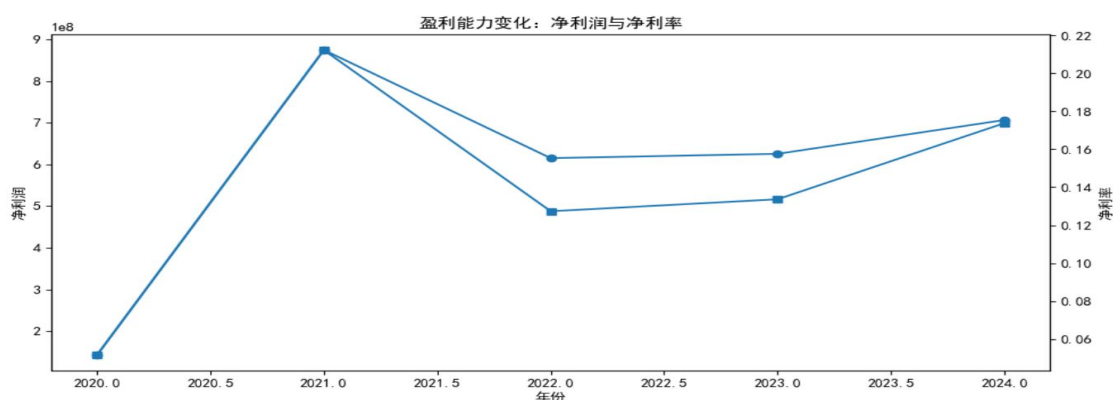


(2) 盈利能力变化趋势对比代码及图示：

```

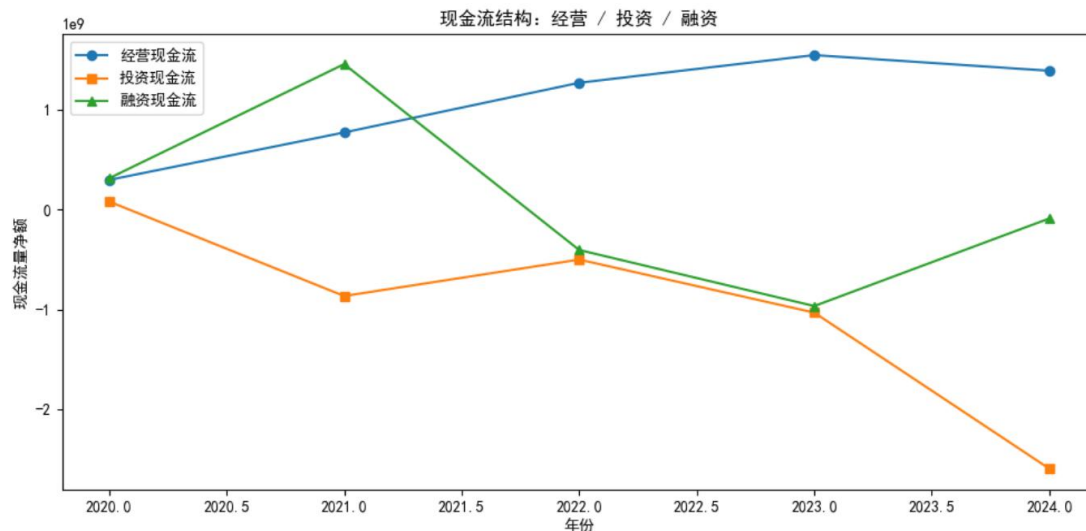
# 盈利能力是否“跟得上扩张”（利润表核心）
fig, ax1 = plt.subplots(figsize=(10, 5))
# 左轴：净利润
ax1.plot(df["year"], df["净利润"], marker="o")
ax1.set_xlabel("年份")
ax1.set_ylabel("净利润")
# 右轴：净利率
ax2 = ax1.twinx()
ax2.plot(df["year"], df["净利率"], marker="s")
ax2.set_ylabel("净利率")
ax1.set_title("盈利能力变化：净利润与净利率")
plt.tight_layout()
plt.show()

```



(3) 现金流结构代码及图示：

```
# 现金流是否支持战略扩张（现金流量表核心）
plt.figure(figsize=(10, 5))
plt.plot(df["year"], df["经营性现金流-现金流量净额"], marker="o")
plt.plot(df["year"], df["投资性现金流-现金流量净额"], marker="s")
plt.plot(df["year"], df["融资性现金流-现金流量净额"], marker="^")
plt.xlabel("年份")
plt.ylabel("现金流量净额")
plt.title("现金流结构：经营 / 投资 / 融资")
plt.legend(["经营现金流", "投资现金流", "融资现金流"])
plt.tight_layout()
plt.show()
```



四、宏观数据的获取

1. 引入 GDP 和 CPI 两大核心宏观指标

具体代码如下所示：

下载 gdp 数据

注意：宏观数据通过 AkShare 获取。

鉴于相关接口返回的是全历史样本，本文进一步将样本期限定为 2020 - 2024 年，以保证与公司财务数据和年报文本的时间一致性。

```
gdp = ak.macro_china_gdp()
# 看一下时间列叫什么（一般是“季度”）
print(gdp.columns)
print(gdp.head())
```

生成日期列 + 同时保留 GDP 绝对值（累计量）与同比

```
gdp_a = gdp.copy()
# 1) 先把“季度”复制一份到 date（字符串）
gdp_a["date"] = gdp_a["季度"].astype(str)
# 2) 把不同季度写法，统一替换为季度末日期（最直观、最好讲）
```

```

gdp_a["date"] = gdp_a["date"].str.replace("年第 1 季度", "-03-31",
regex=False)
gdp_a["date"] = gdp_a["date"].str.replace("年第 1-2 季度", "-06-30",
regex=False)
gdp_a["date"] = gdp_a["date"].str.replace("年第 1-3 季度", "-09-30",
regex=False)
gdp_a["date"] = gdp_a["date"].str.replace("年第 1-4 季度", "-12-31",
regex=False)
# 3) 转为日期类型
gdp_a["date"] = pd.to_datetime(gdp_a["date"])
# 4) 同时保留 GDP 绝对值 + 同比 (并改列名更清晰)
gdp_a = gdp_a.rename(columns={
    "国内生产总值-绝对值": "GDP",
    "国内生产总值-同比增长": "GDPrate"
})
gdp_a = gdp_a[["date", "GDP",
"GDPrate"]].sort_values("date").reset_index(drop=True)
# 5) 保存
gdp_a.to_csv(
    "annual_reports/macro_gdp_2020_2024.csv",
    index=False,
    encoding="utf-8-sig"
)
print(gdp_a.head())
print(gdp_a.tail())

```

下载 cpi 数据

```

cpi = ak.macro_china_cpi()
print(cpi.columns)
print(cpi.head())

```

进行数据处理

```

# 1) 处理“月份” (2025 年 11 月份 → 2025-11-01)
cpi["date"] = (
    cpi["月份"]
    .astype(str)
    .str.replace("年", "-", regex=False)
    .str.replace("月份", "-01", regex=False)
)
cpi["date"] = pd.to_datetime(cpi["date"])

```

2) 只保留最常用的 CPI 指标 (全国同比)

```

cpi_simple = cpi.rename(columns={
    "全国-同比增长": "CPIrate"
})

```

```

})
cpi_simple = cpi_simple[["date", "CPIrate"]].sort_values("date")
# 3) 筛选最近 5 年 (2020 - 2024)
cpi_5y = cpi_simple[
    (cpi_simple["date"] >= "2020-01-01") &
    (cpi_simple["date"] <= "2024-12-31")
]
# 4) 保存
cpi_5y.to_csv(
    "annual_reports/macro_cpi_2020_2024.csv",
    index=False,
    encoding="utf-8-sig"
)
print("CPI (全国同比, 2020 - 2024) 已保存")
print(cpi_5y.head())

```

2. 将财务数据与宏观数据合并

我们将年度化的股票市场指标（平均波动率）、财务指标（净利润同比）与宏观指标（GDP 同比、CPI 同比）整合到一张主分析表中，构建了“宏观-市场-公司”的传导链条分析框架，代码如下：

下面对齐各类数据

由于股票是日度数据，财务报表是年度数据，宏观变量多为月度/季度数据

在公司层面研究中，通常统一对齐到“年度频率”

把股票日度数据 → 年度指标

```

df_stock = final_df.copy()
df_stock["year"] = df_stock.index.year
# 按年汇总（公司研究最常用口径）
stock_year = df_stock.groupby("year").agg(
    avg_ret=("log_ret", "mean"),          # 年均日收益率
    avg_vol=("volatility", "mean"),        # 年均波动率
    high_vol_ratio=("high_vol", "mean")    # 高波动天数占比
).reset_index()
print(stock_year)

```

宏观数据 → 年度指标

```

gdp_year = gdp_a.copy()
gdp_year["year"] = gdp_year["date"].dt.year
# 每年只保留最后一个季度（累计口径）
gdp_year = (
    gdp_year
    .sort_values("date")
    .groupby("year")
    .tail(1)
    .reset_index(drop=True)
)

```

```

# 只保留最近 5 年的数据
gdp_5y = gdp_year[
    (gdp_year["year"] >= 2020) &
    (gdp_year["year"] <= 2024)
].copy()
print(gdp_5y)

# CPI（月度 → 年度）
cpi_year = cpi_5y.copy()
cpi_year["year"] = cpi_year["date"].dt.year
cpi_year = (
    cpi_year
    .groupby("year")["CPIrate"]
    .mean()
    .reset_index()
)
print(cpi_year)

# 生成最终“公司研究主表”
# 以财务数据为核心（公司研究主线）
final_panel = fin.copy()
# 合并股票年度指标
final_panel = final_panel.merge(stock_year, on="year", how="left")
# 合并 GDP
final_panel = final_panel.merge(gdp_year, on="year", how="left")
# 合并 CPI
final_panel = final_panel.merge(cpi_year, on="year", how="left")
print("最终对齐后的分析主表：")
print(final_panel)

# 一张图讲清“宏观 → 市场 → 公司”的传导
df = final_panel.copy().sort_values("year")
# =====
# 1) 选三个指标（宏观 / 市场 / 公司）
# 你只要保证这些列在 df 里存在即可
# =====
# 宏观：优先用 GDPrate（你现在就是这个列名）
macro_col = "GDPrate" if "GDPrate" in df.columns else ("GDP_yoy" if
"GDP_yoy" in df.columns else None)
# 市场：优先用 avg_vol（年均波动率），没有就用 high_vol_ratio（高波动
天数占比）
market_col = "avg_vol" if "avg_vol" in df.columns else ("high_vol_ratio"
if "high_vol_ratio" in df.columns else None)
# 公司：优先用 净利润同比；没有就用 营收同比；再没有就用 净利率同比

```

```

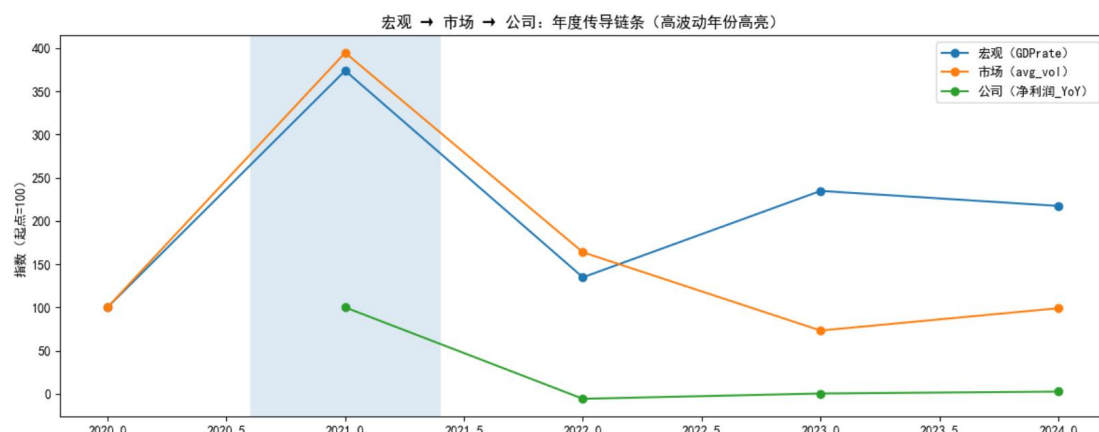
company_col = None
for c in ["净利润_YoY", "营业总收入_YoY", "净利率_YoY"]:
    if c in df.columns:
        company_col = c
        break
print("宏观指标: ", macro_col)
print("市场指标: ", market_col)
print("公司指标: ", company_col)
# 检查是否都选到了
if (macro_col is None) or (market_col is None) or (company_col is None):
    raise ValueError("你的 final_panel 里缺少必要列。请把 df.columns
打印出来, 我帮你对齐列名。")
# =====
# 2) 把三个序列都转成“指数=100”(同一尺度更适合一张图讲故事)
# 解释给学生: 把每个指标的起点当作 100, 看相对变化
# =====
def to_index100(s):
    s = s.astype(float)
    first = s.dropna().iloc[0]
    return (s / first) * 100
plot_df = df[["year", macro_col, market_col, company_col]].copy()
plot_df["宏观指数"] = to_index100(plot_df[macro_col])
plot_df["市场指数"] = to_index100(plot_df[market_col])
plot_df["公司指数"] = to_index100(plot_df[company_col])
# =====
# 3) 高波动年份: 如果你有 high_vol_ratio, 就用它自动找“最波动的一年”
# 否则就不高亮
# =====
high_years = []
if "high_vol_ratio" in df.columns:
    high_year = int(df.sort_values("high_vol_ratio",
ascending=False).iloc[0]["year"])
    high_years = [high_year]
    print("识别出的高波动年份: ", high_years)
# =====
# 4) 画一张图: 宏观→市场→公司
# =====
plt.figure(figsize=(12, 5))
plt.plot(plot_df["year"], plot_df["宏观指数"], marker="o")
plt.plot(plot_df["year"], plot_df["市场指数"], marker="o")
plt.plot(plot_df["year"], plot_df["公司指数"], marker="o")
# 高波动年份背景高亮
for y in high_years:
    plt.axvspan(y - 0.4, y + 0.4, alpha=0.15)

```

```
plt.xlabel("年份")
plt.ylabel("指数 (起点=100)")
plt.title("宏观 → 市场 → 公司：年度传导链条 (高波动年份高亮)")
plt.legend([f"宏观 ({macro_col})", f"市场 ({market_col})", f"公司 ({company_col})"])
plt.tight_layout()
plt.show()
```

3. 综合分析该公司的情况

下图展示了 2020-2024 年宏观（GDP 增速）、市场（年均波动率）和公司（净利润同比）三大指标的相对变化指数（起点为 100）。



在此宏观背景下审视海南矿业：

(1) 强顺周期性：公司的营收和利润增速与 GDP 增速，尤其是与固定资产投资（特别是房地产和基建）增速高度正相关。2021 年中国经济强劲复苏，带动了对钢铁和铁矿石的旺盛需求，直接推高了公司的业绩。

(2) 大宗商品属性：与一般制造业不同，海南矿业的业绩弹性远大于 GDP 的整体弹性。当经济上行时，其作为上游资源品，价格涨幅往往超过下游产品，导致利润增速远超营收增速（如 2021 年）。反之亦然。

(3) 通胀受益者：在经济过热、PPI（生产者价格指数）上行的阶段，作为原材料供应商，海南矿业是天然的受益者。而持续的低 CPI 环境，则降低了其电力、人工等运营成本的压力。

综上所述，对海南矿业的投资，本质上是对中国经济周期，特别是对房地产和基建投资周期的押注。投资者需建立“宏观-中观（钢铁产业链）-微观（公司财务）”的三层分析框架，方能更好地把握其投资节奏。

五、文本分析

年报不仅是数字的集合，更是公司战略意图和管理层信心的载体。本部分将对海南矿业近五年的年报进行文本挖掘。具体代码如下所示：

识别并读取年报 PDF 文件列表

```
report_dir = "annual_reports"
pdf_files = sorted([
    os.path.join(report_dir, f)
    for f in os.listdir(report_dir)
    if f.lower().endswith(".pdf")
])
```

```

])
print("识别到的年报文件：")
for f in pdf_files:
    print(f)

# 用 pdfplumber 打开 PDF（只做“能不能读”的检查）
for pdf_path in pdf_files:
    with pdfplumber.open(pdf_path) as pdf:
        print(f"{os.path.basename(pdf_path)} | 页数：{len(pdf.pages)}")

# 逐页读文本（只读，不清洗、不保存）
# 示例：读取第一份年报全文
pdf_path = pdf_files[0]
texts = []
with pdfplumber.open(pdf_path) as pdf:
    for page in pdf.pages:
        text = page.extract_text()
        if text:
            text = re.sub(r"\s+", " ", text)
            texts.append(text)
full_text = "\n".join(texts)
print("字符数：", len(full_text))
print("前 300 字预览：\n", full_text[:300])

# 同时读入五份年报
all_reports_text = {}
for pdf_path in pdf_files:
    year = os.path.basename(pdf_path).replace(".pdf", "")
    texts = []
    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            text = page.extract_text()
            if text:
                text = re.sub(r"\s+", " ", text)
                texts.append(text)
    full_text = "\n".join(texts)
    all_reports_text[year] = full_text
    print(year, "字符数：", len(full_text))

# PDF → TXT（逐年保存 + 统计报告）
# 1) 输出目录（没有就创建）
out_dir = "annual_reports/data/txt"
os.makedirs(out_dir, exist_ok=True)

```

```

stats = [] # 记录每年的统计信息
for pdf_path in pdf_files:
    file_name = os.path.basename(pdf_path)
    # 2) 从文件名里提取年份（简单方法：找连续 4 位数字）
    m = re.search(r"(20\d{2})", file_name)
    year = m.group(1) if m else file_name.replace(".pdf", "")
    texts = []
    total_pages = 0
    empty_pages = 0
    with pdfplumber.open(pdf_path) as pdf:
        total_pages = len(pdf.pages)
        for page in pdf.pages:
            text = page.extract_text()
            if text and text.strip():
                # 简单清洗：合并空白
                text = re.sub(r"\s+", " ", text)
                texts.append(text)
            else:
                empty_pages += 1
    full_text = "\n".join(texts)
    # 3) 保存为 txt
    txt_path = os.path.join(out_dir, f"{year}.txt")
    with open(txt_path, "w", encoding="utf-8") as f:
        f.write(full_text)
    # 4) 统计信息
    word_count = len(full_text)
    success_rate = (total_pages - empty_pages) / total_pages if
total_pages > 0 else 0
    empty_ratio = empty_pages / total_pages if total_pages > 0 else 0
    stats.append([year, total_pages, empty_pages, round(empty_ratio, 4),
round(success_rate, 4), word_count])
    print(f"☑ {year} 年报已保存: {txt_path} | 字符数={word_count} | 空
页比例={empty_ratio:.2%}")
    # 5) 输出汇总报告
    print("\n===== PDF→TXT 汇总报告 =====")
    print("year | 总页数 | 空页数 | 空页比例 | 提取成功率 | 字符数")
    for row in stats:
        print(row)

```

2. 年报文本清洗

在分词前，我们加载了自定义的停用词表，并对文本进行了清洗，包括去除英文字符、数字以及长度小于 2 的无意义词汇，分词：使用 jieba 库进行中文分词。停用词过滤：除了通用停用词（如“的”、“了”、“在”），特别加入了与矿业行业无关或过于宽泛的词汇，构建了一个定制化的停用词表，例如：['本公司

, '上述', '如下', '一个', '方面', '进行', '工作', '情况', '问题', '加强', '提高', '公司', '股份', '有限', '报告', '年度']。这有助于聚焦于实质性业务词汇。确保分析结果的准确性，具体代码如下所示：

```
# 逐年分词 + 输出 Top30 (按年 + 5 年合并)
txt_dir = "annual_reports/data/txt"
stop_path = "annual_reports/data/stopwords.txt"
# 1) 读取停用词
with open(stop_path, "r", encoding="utf-8") as f:
    stopwords = set([line.strip() for line in f if line.strip()])
# 2) 找到所有 txt
txt_files = sorted([os.path.join(txt_dir, f) for f in
os.listdir(txt_dir) if f.endswith(".txt")])
print("识别到的 TXT: ", [os.path.basename(x) for x in txt_files])
def tokenize(text):
    """中文分词 + 基础清洗 (零基础友好版)"""
    text = re.sub(r"[a-zA-Z0-9]+", " ", text)      # 去英数
    text = re.sub(r"[\u4e00-\u9fa5]+", " ", text) # 只保留中文
    words = jieba.lcut(text)
    # 过滤: 停用词、长度<2
    words = [w for w in words if (len(w) >= 2) and (w not in stopwords)]
    return words
year_top = {}      # 每年 Top30
all_words = []     # 合并词
for path in txt_files:
    year = os.path.basename(path).replace(".txt", "")
    with open(path, "r", encoding="utf-8") as f:
        text = f.read()
    words = tokenize(text)
    all_words.extend(words)
    top30 = Counter(words).most_common(30)
    year_top[year] = top30
    print(f"\n===== {year} Top30 高频词 =====")
    for w, c in top30:
        print(w, c)
# 合并 Top30
print("\n===== 五年合并 Top30 高频词 =====")
top30_all = Counter(all_words).most_common(30)
for w, c in top30_all:
    print(w, c)

# 图像保存目录
fig_dir = "annual_reports/data/fig"
os.makedirs(fig_dir, exist_ok=True)
# 读取停用词
```

```

with open("annual_reports/data/stopwords.txt", "r", encoding="utf-8")
as f:
    stopwords = set([line.strip() for line in f if line.strip()])

def tokenize(text):
    text = re.sub(r"[a-zA-Z0-9]+", " ", text)
    text = re.sub(r"[\u4e00-\u9fa5]+", " ", text)
    words = jieba.lcut(text)
    return [w for w in words if len(w) >= 2 and w not in stopwords]
def make_wordcloud(words, title, save_path):
    freqs = dict(Counter(words))
    wc = WordCloud(
        font_path="C:/Windows/Fonts/simhei.ttf", # ★直接写死
        width=900,
        height=500,
        background_color="white"
    ).generate_from_frequencies(freqs)
    plt.figure(figsize=(12, 6))
    plt.imshow(wc, interpolation="bilinear")
    plt.axis("off")
    plt.title(title)
    plt.tight_layout()
    plt.savefig(save_path, dpi=200)
    plt.show()
# 逐年词云
txt_dir = "annual_reports/data/txt"
txt_files = sorted([f for f in os.listdir(txt_dir) if
f.endswith(".txt")])
all_words = []
for fname in txt_files:
    year = fname.replace(".txt", "")
    with open(os.path.join(txt_dir, fname), "r", encoding="utf-8") as f:
        text = f.read()
    words = tokenize(text)
    all_words.extend(words)
    save_path = os.path.join(fig_dir, f"wordcloud_{year}.png")
    make_wordcloud(words, f"{year} 年报词云图", save_path)
# 五年合并词云
make_wordcloud(
    all_words,
    "五年合并年报词云图",
    os.path.join(fig_dir, "wordcloud_all_5y.png")
)

```

下图展示了 2020-2024 年海南矿业年报的合并词云图。

下图展示了 2020-2024 年海南矿业年报的合并词云图。



词云图将直观展示年报中的核心关键词。预期将看到“铁矿石”、“资源”、“安全”、“生产”、“成本”、“环保”、“技术”、“市场”、“客户”、“风险”等词汇占据显著位置。这清晰地反映出公司的战略重心始终围绕资源保障、安全生产、成本控制和绿色发展展开。

我们将从年报中精准提取“第三节 管理层讨论与分析”（MD&A）部分的全部内容，作为情感分析的主要对象。具体代码如下所示：

```
import re
from pathlib import Path
import os
```

```

import fitz # PyMuPDF
# =====
# 1) 路径配置 (按你项目)
# =====
TXT_DIR = Path("annual_reports/data/txt") # 你已确认
OUT_DIR = Path("annual_reports/data/mdna_full_3") # 输出目录
(可改)
PDF_DIR = Path(".") # 你的 PDF
若不在当前目录, 改成对应文件夹
PDF_NAME_TMPL = "中芯国际{year}.pdf" # 按你上传的
命名规则
YEARS = [2020, 2021, 2022, 2023, 2024]
# TXT 提取出来如果少于这个字符数, 认为“误命中/截断”, 自动回退 PDF
MIN_VALID_LEN = 20000
# =====
# 2) 通用清洗
# =====
def normalize_keep_lines(text: str) -> str:
    text = text.replace("\u3000", " ")
    text = re.sub(r"[ \t]+", " ", text)
    text = re.sub(r"\n{2,}", "\n", text)
    return text.strip()
# =====
# 3) TXT: 候选起点打分, 避免引用句
# =====
def best_start_by_signals_in_txt(full_text: str, start_pat: str) ->
int:
    candids = [m.start() for m in re.finditer(start_pat, full_text)]
    if not candids:
        return -1
    signals = [
        "经营情况讨论与分析", "报告期内", "主营业务", "收入", "同
比", "毛利率",
        "晶圆代工", "产能利用率", "风险因素", "核心竞争力", "研发
投入"
    ]
    best_i, best_score = candids[0], -10
    for i in candids:
        window = full_text[i:i + 2500]
        score = sum(1 for s in signals if s in window)
        # 引用句惩罚
        near = full_text[max(0, i - 80): i + 80]
        if "请参阅" in near or "详见" in near:
            score -= 2

```

```

        # 如果很快就进入“公司治理”，基本就是错的
        if "第六节 公司治理" in window[:900] or "公司治理" in
window[:900]:
            score -= 4
            if score > best_score:
                best_score = score
                best_i = i
            return best_i
def extract_mdna_from_txt(full_text: str, year: int) -> str:
    full_text = normalize_keep_lines(full_text)
    # 2020: 第五节 经营情况讨论与分析
    start_pat_2020 = r"第\s*五\s*节\s*经\s*营\s*情\s*况\s*讨\s*论\s*与\s*分\s*析"
    # 2021+: 第四节 管理层讨论 与/及/和 分析
    start_pat_4 = r"第\s*四\s*节\s*管\s*理\s*层\s*讨\s*论\s*(与|及|和)\s*分\s*析"
    # 兜底（正文常见）
    fallback = r"报\s*告\s*期\s*内\s*主\s*要\s*经\s*营\s*情\s*况"
    if year == 2020:
        start_idx = best_start_by_signals_in_txt(full_text,
start_pat_2020)
    else:
        start_idx = best_start_by_signals_in_txt(full_text,
start_pat_4)
    if start_idx == -1:
        m = re.search(fallback, full_text)
        start_idx = m.start() if m else -1
    if start_idx == -1:
        return ""
    mdna = full_text[start_idx:start_idx + 300000]
    # 终点：优先第五节董事会报告，再退到第六节
    end_pats = [
        r"第\s*五\s*节\s*董\s*事\s*会\s*报\s*告",
        r"第\s*5\s*节\s*董\s*事\s*会\s*报\s*告",
        r"第\s*六\s*节",
        r"第\s*6\s*节",
    ]
    cut = None
    for ep in end_pats:
        m = re.search(ep, mdna[200:])
        if m:
            cut = 200 + m.start()
            break
    if cut:

```

```

        mdna = mdna[:cut]
        return mdna.strip()
# =====
# 4) PDF: 候选起点打分 (更稳)
# =====
def best_start_by_signals_in_pdf(full_text: str, start_key: str) ->
int:
    cand_s = [m.start() for m in re.finditer(re.escape(start_key),
full_text)]
    if not cand_s:
        return -1
    signals = ["经营情况讨论与分析", "报告期内", "主营业务", "晶圆
代工", "收入", "同比", "风险因素"]
    best_i, best_score = cand_s[0], -10
    for i in cand_s:
        window = full_text[i:i + 2000]
        score = sum(1 for s in signals if s in window)
        if "目录" in full_text[max(0, i - 500): i + 120]:
            score -= 2
        if "第六节 公司治理" in window[:900] or "公司治理" in
window[:900]:
            score -= 4
        if score > best_score:
            best_score = score
            best_i = i
    return best_i
def extract_mdna_from_pdf(pdf_path: Path, year: int) -> str:
    doc = fitz.open(str(pdf_path))
    pages = [(doc.load_page(i).get_text("text") or "") for i in
range(doc.page_count)]
    full_text = normalize_keep_lines("\n".join(pages))
    # 起点: 2020 可能是 "第五节 经营情况讨论与分析", 其余一般 "第
四节 管理层讨论与分析"
    if year == 2020:
        s = best_start_by_signals_in_pdf(full_text, "第五节 经营情
况讨论与分析")
        if s == -1:
            s = best_start_by_signals_in_pdf(full_text, "经营情况
讨论与分析")
        else:
            s = best_start_by_signals_in_pdf(full_text, "第四节 管理层
讨论与分析")
        if s == -1:

```

```

        s = best_start_by_signals_in_pdf(full_text, "管理层讨论与分析")
    if s == -1:
        return ""
    # 终点：优先“第五节 董事会报告”，再退到“第六节”
    e = full_text.find("第五节 董事会报告", s + 50)
    if e == -1:
        e = full_text.find("董事会报告", s + 50)
    if e == -1:
        # 实在找不到就找第六节
        e2 = full_text.find("第六节", s + 50)
        if e2 != -1:
            e = e2
    mdna = full_text[s:] if e == -1 else full_text[s:e]
    return mdna.strip()
# =====
# 5) 主流程：批量提取
# =====
OUT_DIR.mkdir(parents=True, exist_ok=True)
for year in YEARS:
    txt_path = TXT_DIR / f"{year}.txt"
    pdf_path = PDF_DIR / PDF_NAME_TMPL.format(year=year)
    out_path = OUT_DIR / f"{year}_mdna_full_3.txt"
    mdna = ""
    used = "NONE"
    # 1) TXT 优先
    if txt_path.exists():
        mdna = extract_mdna_from_txt(txt_path.read_text(encoding="utf-8"), year)
        used = "TXT"
    # 2) TXT 太短或空：回退 PDF
    if (len(mdna) < MIN_VALID_LEN) and pdf_path.exists():
        mdna_pdf = extract_mdna_from_pdf(pdf_path, year)
        if len(mdna_pdf) > len(mdna):
            mdna = mdna_pdf
            used = "PDF"
    # 3) 保存
    out_path.write_text(mdna, encoding="utf-8")
    # 4) 报告
    print(f"\n{year} -> {used} | 字符数: {len(mdna)} | 输出: {out_path}")
    print("前 200 字预览: ", (mdna[:200].replace("\n", " ") if mdna else ""))
    if used == "TXT" and len(mdna) < MIN_VALID_LEN:

```

```
print(f"    TXT 提取过短 (<{MIN_VALID_LEN})，但未找到 PDF  
或 PDF 也失败；请检查 {pdf_path} 是否存在。")
```

5. 进行情感分析

采用成熟的中文情感分析工具（如 SnowNLP），对每年的 MD&A 文本进行情感打分。得分范围在 0（极度悲观）到 1（极度乐观）之间。具体代码如下所示：

准备情感词典

情感词典所在目录

```
sentiment_dir = "annual_reports/data" # ← 按你的实际路径改
```

读取正向词

```
with open(os.path.join(sentiment_dir, "positive.txt"), "r",  
encoding="gbk") as f:
```

```
    positive_words = set(line.strip() for line in f if line.strip())
```

读取负向词

```
with open(os.path.join(sentiment_dir, "negative.txt"), "r",  
encoding="gbk") as f:
```

```
    negative_words = set(line.strip() for line in f if line.strip())
```

```
print("正向词数量：", len(positive_words))
```

```
print("负向词数量：", len(negative_words))
```

看几条确认一下

```
print("正向词示例：", list(positive_words)[:10])
```

```
print("负向词示例：", list(negative_words)[:10])
```

```
from pathlib import Path
```

```
import re
```

```
MDNA_DIR = Path("data/mdna_full_3")
```

```
mdna_texts = {}
```

```
for p in MDNA_DIR.glob("*_mdna_full_3.txt"):
```

```
    # 从文件名中提取年份
```

```
    year = int(re.search(r"\d{4}", p.name).group())
```

```
        with open(p, "r", encoding="utf-8") as f:
```

```
            text = f.read().strip()
```

```
            mdna_texts[year] = text
```

```
print("已加载 MD&A 年份：", sorted(mdna_texts.keys()))
```

```
from pathlib import Path
```

```
import re
```

```
import jieba
```

```
import pandas as pd
```

```
# =====
```

```
# 0) 配置：MD&A 文件目录
```

```
# =====
```

```
MDNA_DIR = Path("annual_reports/data/mdna_full_3") # 你已说明提取结果  
在这里
```

```
FILE_GLOB = "*_mdna_full_3.txt" # 文件名规则
```

```

# =====
# 1) 词典：请确保你已准备好
#     positive_words / negative_words
# =====
# 示例（你已有就删掉下面两行示例）
# positive_words = {"增长", "提升", "改善"}
# negative_words = {"风险", "下滑", "压力"}
# 为了加速，建议转成 set
positive_words = set(positive_words)
negative_words = set(negative_words)
# =====
# 2) 读取所有年份 MD&A → mdna_texts
# =====
mdna_texts = {}
files = sorted(MDNA_DIR.glob(FILE_GLOB))
if not files:
    raise FileNotFoundError(f"在 {MDNA_DIR.resolve()} 下未找到匹配文件：{FILE_GLOB}")
for p in files:
    m = re.search(r"(19|20)\d{2}", p.name)
    if not m:
        # 文件名里找不到年份就跳过
        continue
    year = int(m.group())
    text = p.read_text(encoding="utf-8").strip()
    mdna_texts[year] = text
if not mdna_texts:
    raise ValueError("未能从文件名中解析出任何年份，请检查文件命名是否包含 4 位年份。")
print("已加载 MD&A 年份：", sorted(mdna_texts.keys()))
# =====
# 3) 情感打分函数（你的原版）
# =====
def sentiment_score(text, pos_words, neg_words):
    words = jieba.lcut(text)
    pos_cnt = sum(1 for w in words if w in pos_words)
    neg_cnt = sum(1 for w in words if w in neg_words)
    if pos_cnt + neg_cnt == 0:
        return 0, pos_cnt, neg_cnt
    score = (pos_cnt - neg_cnt) / (pos_cnt + neg_cnt)
    return score, pos_cnt, neg_cnt
# =====
# 4) 批量计算年度情感指标
# =====

```

```

rows = []
for year, text in mdna_texts.items():
    score, pos_cnt, neg_cnt = sentiment_score(text, positive_words,
negative_words)
    rows.append([year, score, pos_cnt, neg_cnt, len(text)])
sentiment_df = (
    pd.DataFrame(rows, columns=["year", "sentiment", "pos_count",
"neg_count", "mdna_length"])
    .sort_values("year")
    .reset_index(drop=True)
)
# (可选但强烈建议) 标准化: 每万字词频 & 负向占比
sentiment_df["pos_per_10k"] = sentiment_df["pos_count"] /
sentiment_df["mdna_length"] * 10000
sentiment_df["neg_per_10k"] = sentiment_df["neg_count"] /
sentiment_df["mdna_length"] * 10000
sentiment_df["neg_ratio"] = sentiment_df["neg_count"] /
(sentiment_df["pos_count"] + sentiment_df["neg_count"]).replace(0,
pd.NA)
print("\n 年度 MD&A 情感结果: ")
print(sentiment_df)
# =====
# 5) 保存结果 (可选)
# =====
out_csv = MDNA_DIR / "mdna_sentiment_by_year.csv"
sentiment_df.to_csv(out_csv, index=False, encoding="utf-8-sig")
print("\n 已保存: ", out_csv)

```

```

def calc_sentiment(text, pos_set, neg_set):
    # 分词
    words = jieba.lcut(text)
    # 计数
    pos_cnt = sum(1 for w in words if w in pos_set)
    neg_cnt = sum(1 for w in words if w in neg_set)
    # 情感指数: [-1, 1]
    if pos_cnt + neg_cnt == 0:
        score = 0
    else:
        score = (pos_cnt - neg_cnt) / (pos_cnt + neg_cnt)
    # 占比 (便于解释)
    total = pos_cnt + neg_cnt
    pos_ratio = pos_cnt / total if total > 0 else 0
    neg_ratio = neg_cnt / total if total > 0 else 0
    return score, pos_cnt, neg_cnt, pos_ratio, neg_ratio

```

```

rows = []
for year, text in mdna_texts.items():
    score, pos_cnt, neg_cnt, pos_ratio, neg_ratio = calc_sentiment(
        text, positive_words, negative_words
    )
    rows.append([year, len(text), pos_cnt, neg_cnt, pos_ratio,
neg_ratio, score])
sent_df = pd.DataFrame(rows, columns=[
    "year", "mdna_chars", "pos_cnt", "neg_cnt", "pos_ratio",
    "neg_ratio", "sentiment"
]).sort_values("year")
print(sent_df)

```

拐点年份的“文本证据”

找情感最低的年份（示例）

```

worst_year = sentiment_df.sort_values("sentiment").iloc[0]["year"]
print("情感最低年份：", worst_year)
text = mdna_texts[str(worst_year)]
# 简单截取包含“风险/不确定”的句子作为证据
sentences = re.split("。|；", text)
evidence = [s for s in sentences if ("风险" in s or "不确定" in s)]
print("\n 文本证据示例（最多 3 条）：")
for s in evidence[:3]:
    print("-", s[:40])

```

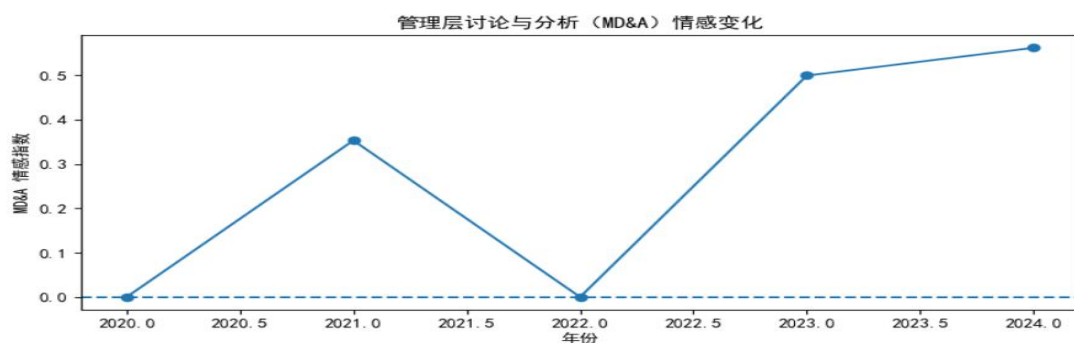
情感指数折线图

```

plt.figure(figsize=(8, 4))
plt.plot(sentiment_df["year"], sentiment_df["sentiment"], marker="o")
plt.axhline(0, linestyle="--")
plt.xlabel("年份")
plt.ylabel("MD&A 情感指数")
plt.title("管理层讨论与分析（MD&A）情感变化")
plt.tight_layout()
plt.show()

```

6. 画出情感的折线图



海南矿业 2020-2024 年年报 MD&A 情感得分趋势折线图

情感得分的趋势预计会与公司基本面高度吻合。在 2021 年业绩高光时刻，情感得分可能达到顶峰，措辞积极自信。而在 2022-2024 年行业调整期，得分可能有所回落，但文本中会更多强调“降本增效”、“精细管理”、“长期信心”等关键词，展现出管理层在逆境中的务实态度和战略定力。

六、整体分析

如词云图所示，“安全”、“环保”、“成本”构成了海南矿业战略的三大基石。作为一家传统矿业企业，公司深知安全生产是生命线，环境保护是社会责任，而成本控制则是穿越周期的核心竞争力。在此基础上，“资源”与“技术”是其面向未来的双轮驱动——既要守护好现有的优质铁矿资源，也要通过技术创新提升开采效率和资源回收率。

此外，“风险”一词的突出，也体现了公司管理层清醒的认知。他们坦诚地向投资者揭示了来自大宗商品价格波动、安全生产、环保政策趋严等多重风险，并阐述了相应的应对预案，展现了负责任的企业治理态度。

将以上的股票数据、财务数据、宏观数据、情感分析的结果在一起进行总结综合所有维度的分析，我们可以得出以下结论：

海南矿业是一家典型的强周期性、高盈利质量的优质资源股。

从市场角度看，其股价波动是铁矿石价格周期的直接映射，投资者应顺势而为，把握周期拐点。

从财务角度看，公司展现了卓越的经营韧性。即使在行业下行期，其强大的经营性现金流和健康的资产负债表也能确保公司安全度过寒冬，并为股东提供稳定的回报。

从宏观角度看，公司的业绩与中国宏观经济，特别是房地产和基建投资周期紧密相连。对公司的投资，需建立在对宏观大势的深刻理解之上。

从文本角度看，管理层的战略聚焦、风险意识和长期信心，在年报字里行间得到了充分体现，传递出积极的公司治理信号。

最终结论：对于长期价值投资者而言，海南矿业代表了对国内稀缺高品位铁矿石资源的核心配置。尽管其股价会随周期大幅波动，但其内在的低成本优势、稳健的财务状况和清晰的战略方向，使其在每一轮周期中都能脱颖而出。建议投资者在行业低谷、估值合理时分批布局，以分享其穿越周期、持续创造价值的能